Blood Donor Data Management System

Manual Deployment Guide — DynamoDB, Lambda, API Gateway & S3 Frontend

Console-driven setup (no CloudFormation) — deploy each service by hand, then push your code to GitHub

August 09, 2026

Table of Contents

1. Overview

This guide deploys the same four pieces every time: a DynamoDB table for donor records, a single Python Lambda function handling all CRUD operations, an API Gateway REST API that routes HTTP requests to that Lambda, and a React frontend hosted as a static website on S3. Everything is done through the AWS Console (with CLI equivalents where useful) — no CloudFormation, SAM, or CDK. Once everything works, the last step pushes your project folder to GitHub so it's version-controlled.

Piece	Service	What it does
Database	DynamoDB	Stores donor records (table: Donors)
Backend	Lambda (Python 3.12)	One function, routes by HTTP method + path
API	API Gateway (REST API)	Exposes /donor and /donor/{donorId} over HTTPS
Frontend	S3 static website	Hosts the built React app

2. Prerequisites

- An AWS account with console access to DynamoDB, Lambda, API Gateway, and S3.
- AWS CLI v2 installed and configured, if you want to use the CLI commands alongside console steps.
- Node.js 18+ and npm, to build the React frontend.
- A GitHub account and an empty repository to push this project into.
- The project files: `lambda_function.py` and the `frontend/` React app (provided alongside this PDF).

3. Step 1 — Create the DynamoDB Table

1. Open the AWS Console → search for and open **DynamoDB**.
2. In the left sidebar click **Tables** → **Create table**.
3. **Table name:** `Donors`
4. **Partition key:** `donorId` — type **String**. Leave sort key blank.
5. Under **Table settings**, choose **Customize settings**.
6. **Capacity mode:** select **On-demand** (pay-per-request, no capacity planning needed).
7. Leave encryption at the default (AWS owned key) unless your organization requires otherwise.
8. Scroll down and click **Create table**. Wait for status to change from `Creating` to `Active`.
9. (Recommended) Open the new table → **Backups** tab → enable **Point-in-time recovery** so donor data can be restored if something goes wrong.

Equivalent AWS CLI

```
aws dynamodb create-table \
  --table-name Donors \
  --attribute-definitions AttributeName=donorId,AttributeType=S \
  --key-schema AttributeName=donorId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1

aws dynamodb update-continuous-backups \
  --table-name Donors \
  --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true \
  --region us-east-1
```

4. Step 2 — Create the Lambda Function

1. Open the AWS Console → search for and open **Lambda**.
2. Click **Create function** → **Author from scratch**.
3. **Function name:** e.g. `blood-donor-function`
4. **Runtime: Python 3.12**
5. **Architecture:** x86_64 (default is fine).
6. Under **Permissions**, expand **Change default execution role**. If you already have an IAM role named `lambda role`, choose **Use an existing role** and select it. Otherwise choose **Create a new role with basic Lambda permissions** and name it `lambda role` so it's easy to find later. Click **Create function**.
7. In the **Code** tab, delete the placeholder code in `lambda_function.py` and paste in the full contents of the provided `lambda_function.py` file.
8. Click **Deploy** to save the code.

Set the environment variable

1. Go to the **Configuration** tab → **Environment variables** → **Edit** → **Add environment variable**.
2. Key: `DONORS_TABLE_NAME` — Value: `Donors` — Save.

Grant DynamoDB permissions to the lambda role

1. Still in **Configuration** → **Permissions**, click the role link next to **Role name** — it should say `lambda role` — to open it in IAM.
2. Click **Add permissions** → **Create inline policy**.
3. Service: DynamoDB. Actions: `PutItem`, `GetItem`, `UpdateItem`, `DeleteItem`, `Scan`.
4. Resources: specify the ARN of your Donors table — `arn:aws:dynamodb:<region>:<account-id>:table/Donors` (find your account ID in the console's top-right account menu).
5. Name the policy (e.g. `DonorsTableAccess`) and click **Create policy**.

Note: Without this policy attached to the lambda role, every Lambda call will fail with an AccessDeniedException even though the function code and API Gateway wiring are correct.

Increase timeout (optional but recommended)

Configuration → General configuration → Edit → set Timeout to 10 seconds (the 3-second default is usually fine, but gives more headroom for cold starts).

5. Step 3 — Create the API Gateway REST API

1. Open the AWS Console → search for and open **API Gateway**.
2. Click **Create API** → find **REST API** (not private) → **Build**.
3. **API name:** e.g. `blood-donor-api`. Endpoint type: Regional. Click **Create API**.

3.1 Create the /donor resource

1. In the Resources tree, select the root `/`, click **Create resource**.
2. Resource path: `donor` (no slash, no braces). Click **Create resource**.

3.2 Add methods on /donor (list + create)

1. Select `/donor`, click **Create method**.
2. Method: **GET**. Integration type: **Lambda function**. **Check "Use Lambda Proxy integration"** — this is essential.
3. Enter your Lambda function name, click **Create method**, and accept the permission prompt (this lets API Gateway invoke the Lambda).
4. Repeat for **POST** (create donor), **PUT** and **DELETE** are not needed here since those operate on a single donor (see 3.3).

Gotcha: "Use Lambda Proxy integration" must be checked at creation time. If you forget it, the console will let you create the method anyway, and the symptom shows up later as a response body containing the whole {statusCode, headers, body} object instead of a clean JSON response, with your Lambda seeing httpMethod as None. Fix by deleting the method and recreating it with the box checked — it generally cannot be toggled after creation.

3.3 Create the /donor/{donorId} resource and its methods

1. Select `/donor`, click **Create resource** again.
2. Resource path: type it with braces included — `{donorId}` exactly (case-sensitive). Click **Create resource**.
3. Select `{donorId}`, click **Create method** → **GET** → Lambda function, proxy integration checked, same function. Create, accept permission prompt.
4. Repeat to add **PUT** (update donor) and **DELETE** (remove donor) on the same resource, same Lambda function each time.

Gotcha: The path parameter name is case-sensitive and must exactly match what the Lambda code reads. lambda_function.py reads event['pathParameters']['donorId'] — if you create the resource as {donorID} or {id} instead, the lookup silently returns None and every request to a specific donor will 400 with "donorId is required in the path".

3.4 Enable CORS (so the browser-hosted React app can call this API)

1. Select `/donor`, click **Enable CORS** (or manually add an OPTIONS method — Lambda proxy integrations don't get CORS automatically).
2. Accept the defaults (Access-Control-Allow-Origin: *) and click **Save**.
3. Repeat **Enable CORS** on `{donorId}`.

3.5 Deploy the API

1. Click **Deploy API** (top right or under Actions).
2. Deployment stage: **New stage** — name it `dev`. Click **Deploy**.
3. Copy the **Invoke URL** shown at the top of the Stages page — it looks like `https://abc123.execute-api.us-east-1.amazonaws.com/dev`. You'll need this for the frontend.

Note: Any time you add or change a resource/method afterward, you must click Deploy API again — changes are not live until redeployed, even though the console makes them look saved.

API route summary

Method	Path	Purpose
GET	/donor	List all donors (optional ?bloodGroup=)
POST	/donor	Create a new donor
GET	/donor/{donorId}	Get one donor
PUT	/donor/{donorId}	Update a donor
DELETE	/donor/{donorId}	Delete a donor

6. Step 4 — Build and Deploy the Frontend to S3

4.1 Create the S3 bucket

1. Open the AWS Console → **S3** → **Create bucket**.
2. Bucket name must be globally unique, e.g. `blood-donor-frontend-<your-account-id>`.
3. Under **Object Ownership**, keep ACLs disabled (default).
4. Under **Block Public Access settings**, **uncheck** "Block all public access" and acknowledge the warning — this bucket needs to be publicly readable to serve the website.
5. Click **Create bucket**.

4.2 Enable static website hosting

1. Open the bucket → **Properties** tab → scroll to **Static website hosting** → **Edit**.
2. Enable it. Hosting type: **Host a static website**.
3. Index document: `index.html`
4. Error document: `index.html` (so React Router's client-side routes still work when someone refreshes on a non-root URL).
5. Save. Note the **Bucket website endpoint** URL shown afterward.

4.3 Add a bucket policy for public read access

Go to the **Permissions** tab → **Bucket policy** → **Edit** → paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::blood-donor-frontend-<your-account-id>/*"
    }
  ]
}
```

Replace the bucket name with your actual bucket name, then click **Save changes**.

4.4 Point the frontend at your API and build it

```
cd frontend
cp .env.example .env
# edit .env and set:
# VITE_API_URL=https://abc123.execute-api.us-east-1.amazonaws.com/dev

npm install
npm run build
```

4.5 Upload the build to S3

```
aws s3 sync dist/ s3://blood-donor-frontend-<your-account-id>/ --delete
```

Or drag-and-drop the contents of the `frontend/dist` folder into the bucket via the S3 console (Upload → Add files/folder).

Open the Bucket website endpoint URL from step 4.2 — you should see the donor registry app, able to list, add, edit, and delete donors.

7. Step 5 — Push Your Code to GitHub

1. Create a new, empty repository on GitHub (do not initialize with a README, to avoid a merge conflict).
2. From your project's root folder (containing lambda_function.py and frontend/), initialize git:

```
cd blood-donor-system
git init
git add .
git commit -m "Initial commit: blood donor management system"
git branch -M main
git remote add origin https://github.com/<your-username>/<your-repo>.git
git push -u origin main
```

Note: Make sure frontend/.env and frontend/node_modules are listed in .gitignore before committing — .env may contain environment-specific values you don't want in version control, and node_modules is large and regenerable from package.json.

```
# .gitignore
node_modules/
frontend/dist/
frontend/.env
__pycache__/
*.pyc
.DS_Store
*.zip
```

8. End-to-End Test

Exercise every endpoint directly with curl to confirm the backend independent of the frontend:

```
API=https://abc123.execute-api.us-east-1.amazonaws.com/dev

# Create a donor
curl -X POST "$API/donor" -H "Content-Type: application/json" -d '{
  "fullName": "Asha Rai",
  "bloodGroup": "O+",
  "phone": "+977-9800000000",
  "email": "asha@example.com"
}'

# List donors
curl "$API/donor"

# Filter by blood group
curl "$API/donor?bloodGroup=O%2B"

# Get one donor
curl "$API/donor/<donorId>"

# Update a donor
curl -X PUT "$API/donor/<donorId>" -H "Content-Type: application/json" \
  -d '{"notes": "Second donation"}'

# Delete a donor
curl -X DELETE "$API/donor/<donorId>"
```

9. Common Gotchas

"Missing Authentication Token" on /donor/{id} but /donor works

No child resource exists for the specific-donor path, or the API wasn't redeployed after adding it. Create the {donorId} resource under /donor with GET/PUT/DELETE methods, then Deploy API again.

Response body contains the whole {statusCode, headers, body} object

Lambda Proxy integration isn't enabled on that method. Delete and recreate the method with "Use Lambda Proxy integration" checked.

Donor-specific requests always 400 with "donorId is required in the path"

The resource's path parameter name doesn't match what the code reads. It must be exactly {donorId} — check case.

CORS errors in the browser console

Enable CORS on both /donor and /donor/{donorId} resources in API Gateway, and redeploy. Lambda proxy integrations do not add CORS headers to OPTIONS automatically — only your actual GET/POST/PUT/DELETE responses include the headers your Lambda code sets.

Lambda returns 500 / AccessDeniedException in CloudWatch logs

The lambda role is missing DynamoDB permissions on the Donors table. See Step 2's "Grant DynamoDB permissions" section.

403 Forbidden opening the S3 website URL

Either "Block all public access" is still enabled on the bucket, or the bucket policy wasn't saved. Also check for an account-level S3 Block Public Access setting, which overrides the bucket-level setting.

Frontend loads but shows no donors / network errors

VITE_API_URL in frontend/.env doesn't match the actual Invoke URL, or the build wasn't rebuilt after changing .env (Vite bakes env vars in at build time, not runtime — always run npm run build again after editing .env).

Appendix A — Donor Record Schema

Each item in the DynamoDB Donors table (partition key: `donorId`):

Field	Type	Notes
donorId	String	Partition key, generated server-side (UUID)
fullName	String	Required
bloodGroup	String	Required; one of A+, A-, B+, B-, AB+, AB-, O+, O-
phone	String	Required
email	String	Optional
donationDate	String (ISO date)	Optional
notes	String	Optional
createdAt	String (ISO datetime)	Set on create
updatedAt	String (ISO datetime)	Set on create and every update

Appendix B — Project File Tree

```
blood-donor-system/
■■■ .gitignore
■■■ lambda_function.py          # single Lambda, handles all CRUD routes (paste into console)
■■■ frontend/
    ■■■ package.json
    ■■■ vite.config.js
    ■■■ index.html
    ■■■ .env.example
    ■■■ src/
        ■■■ main.jsx
        ■■■ App.jsx
        ■■■ api.js
        ■■■ index.css
        ■■■ components/
            ■■■ Navbar.jsx
            ■■■ DonorList.jsx
            ■■■ DonorForm.jsx
```